Multi Vendor Earning Blog Site with AI

M.ShahzaibKaramat

April 2026

Department of Computer Science
Lahore Leads Universiy
Lahore – PAKISTAN

Submission Form for Final-Year

PROJECT REPORT

PROJECT ID	52	NUMBER OF MEMBERS	3

TITLE	Multi Vendor Earning Blog Site with AI
SUPERVISOR NAME	

MEMBER NAME	REG. NO.	EMAIL ADDRESS
M. Shahzaib Karamat		
M. Akhlaq Raza		
Abdullah Farooq		

CHECKLIST:

Number of pages in this report	
I/We have enclosed the soft-copy along-with the codes and scripts	YES / NO
My/Our supervisor has attested the attached document	YES / NO
I/We confirm this project is free from plagiarism and misuse of copyrighted material	YES / NO

MEMBERS' SIGNATURES

_____

_____

_____

Supervisor's Signature

Note 1: This paper must be signed by your supervisor
Note 2: Soft-copies of report, source codes, schematics, and executables should be delivered in a CD/DVD

This work, entitled "Multi Vendor Earning Blog Site with AI" has been approved for the award of

Bachelors of Science in Computer Science

Date 12 March 2026

External Examiner: _____

FYP Coordinator: _____

Head of Department: _____

Department of Computer Science

LAHORE LEADS UNIVERSITY

LAHORE– PAKISTAN

Declaration

"No portion of the work referred to in the dissertation has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning".

MEMBERS' SIGNATURES

_____

_____

_____

Acknowledgements

It is usual to thank those individuals who have provided particularly useful assistance, technical or otherwise, during your project.

In the name of God, the most kind and most merciful

We would like to thank our families and friends who kept backing us up at all times, both financially and morally.

We would also like to thank Miss. Tuba Gull for her guidance and for encouraging us to work hard and smart. We found her very helpful while discussing the technical issues in this dissertation work. Her critical comments on our work have made us think of new ideas and techniques in web development, content moderation, and software design.

We are grateful to the God Almighty who provides all the resources of every kind to us, so that we make their proper use for the benefit of mankind. May He keep providing us with all the resources, and the guidance to keep helping the humanity.

Abstract

This project presents a modern blog platform integrated with AI-powered content moderation to ensure safe and responsible publishing. Unlike traditional AI-based systems that generate content, this platform focuses on analyzing user-submitted content in real time to detect harmful, abusive, or policy-violating material. The AI system evaluates content before publication and prevents uploads that may negatively impact individuals or communities.

In addition to content creation, the platform provides monetization opportunities similar to Blogger. Users earn revenue based on article views using a configurable RPM (Rate Per Mille) system. This enables content creators to generate income while maintaining high-quality and policy-compliant content standards enforced by the AI moderation system.

The system includes secure JWT authentication, personalized dashboards for both authors and administrators, structured content management, withdrawal request management, and an announcement system. It is built using Node.js, Express.js, MongoDB, and EJS, ensuring scalability, performance, and maintainability.

Table of Contents

Introduction

CHAPTER 1

1 Introduction 1

1.1 What is the Blog Website? 1

1.2 The Multi-Vendor Concept 1

1.3 Project Goals and Objectives 2

1.4 Technology Stack Overview 3

1.5 Scope of Project 3

1.6 Report Structure 4

CHAPTER 2

2 Literature Review 5

CHAPTER 3

3 Requirements Specification 7

3.1 Functional Requirements 7

3.1.1 Authentication and Content Requirements 7

3.2 Non-Functional Requirements 9

CHAPTER 4

4 Project Design 10

4.1 Use Cases Diagrams 10

4.2 State Diagrams 10

4.3 Sequence Diagrams 10

4.4 Activity Diagrams 11

4.5 Collaborative Diagrams 11

4.6 Class Diagram 11

4.7 UML Diagram 11

4.8 ER Diagram 12

CHAPTER 5

5 Implementation 13

5.1 Tools and Technologies 13

5.2 Design Description 13

5.2.1 New Post Creation Module 13

5.2.2 Blog Analytics & Author Dashboard Module 15

5.2.3 Earnings & Withdrawal Module 16

5.2.4 Admin Dashboard Module 18

5.2.5 Admin Earnings & RPM Management Module 21

5.2.6 Category Management & Announcement Module 23

CHAPTER 6

6 Testing 29

6.1 Unit Testing 29

6.2 Function Testing 29

6.2.1 Testing Requirements - Authentication, Content, Admin, and Earning 29

CHAPTER 7

7 Conclusions & Future Work 32

References 37

Appendix A: List of Components 38

Appendix B: Project Timeline 39

Appendix C: User Manual 41

List of Figures

FIGURE 1-1 Platform Overview - Writer Journey (Register to Create to Moderate to Earn) 1
FIGURE 2-1 MVC Architecture and HTTP Request Lifecycle 5
FIGURE 2-2 Middleware Pipeline - Request Processing Stages 6
FIGURE 3-1 JWT Authentication Flow & Refresh Token Cycle 7
FIGURE 3-2 Activity Diagram - User Registration Flow 8
FIGURE 4-1 Use Case Diagram - Blog Website System 10
FIGURE 4-2 Article State Diagram - Lifecycle 10
FIGURE 4-3 Class Diagram - Mongoose Schema Classes 11
FIGURE 4-4 ER Diagram - MongoDB Collections & Relationships 12
FIGURE 5-1 RBAC - Permission Matrix (Admin vs Normal User) 13
FIGURE 5-2 Security Layers - Defense in Depth (CSP, JWT, RBAC, Validation, AI) 13
FIGURE 5-3 Create Article Form (New Post) - Author Panel 14
FIGURE 5-4 Your Posts Page - Article List with Views & Likes 15
FIGURE 5-5 Blog Analytics Dashboard - Author Panel 16
FIGURE 5-6 Earnings Dashboard - Author Panel 18
FIGURE 5-7 Withdraw Funds Form - Author Panel 19
FIGURE 5-8 Admin Dashboard - Overview Panel 19
FIGURE 5-9 Admin Dashboard - Active Users Trend Chart 21
FIGURE 5-10 Blog Categories - Admin Panel 23
FIGURE 5-11 Contact Messages - Admin Panel 21
FIGURE 5-12 Earnings Overview - Admin Panel with Trend Chart 22
FIGURE 5-13 Withdrawal Requests - Admin Panel 23
FIGURE 5-14 Admin Settings - Password Change 27
FIGURE 5-15 RPM Management - Rate per 1000 Views Configuration 24
FIGURE 5-16 Create Category & Topics Form - Admin 26
FIGURE 5-17 Post a New Announcement - Admin Panel 27
FIGURE 6-1 EJS Server-Side Rendering Pipeline - Controller to Browser 29
FIGURE 7-1 AI Content Moderation Flow - Submission to Approval or Rejection 32
FIGURE 8-1 MongoDB Schema Design & Collection Relationships 34

List of Tables

TABLE 1-1 Technology Stack Overview 3
TABLE 3-1 Functional Requirements 9
TABLE 3-2 Non-Functional Requirements 9
TABLE 5-1 RPM Earning Rate Configuration 27
TABLE 5-2 Category Status Control - Visibility and Earning Rules 28
TABLE 6-1 Form Handling Reference 31
TABLE 6-2 Function Testing Results - All 18 Test Cases PASS 30
TABLE 6-3 Security Testing Checklist - All Critical Checks: DONE 31
TABLE 7-1 User Model Schema - Fields, Types and Constraints 34
TABLE 7-2 Category Model Schema 34
TABLE 7-3 Topic Model Schema 35
TABLE 7-4 Environment Configuration Variables and Security Levels 35
TABLE A-1 npm Package List 38
TABLE B-1 15-Week Project Timeline 40

Chapter 1

1 Introduction

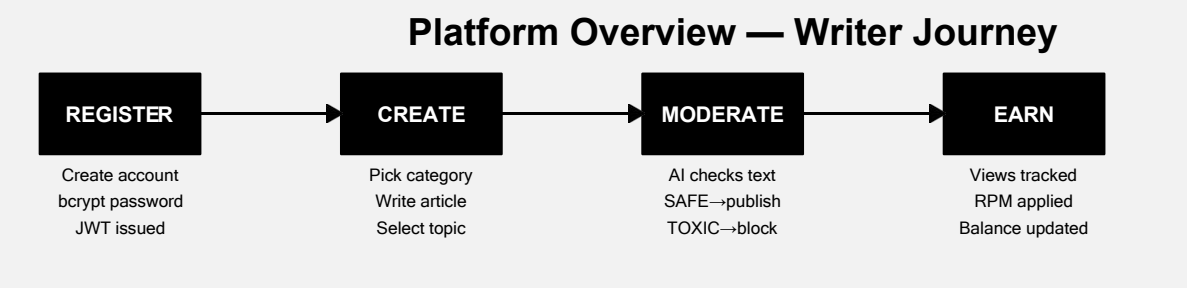


Figure 1-1 Platform Overview — Writer Journey (Register → Create → Moderate → Earn)

This is one of the most important components of the report. The Blog Website is a comprehensive multi-vendor blogging and earning platform where writers register, build their profile, publish articles, and receive monetary compensation based on article views. This chapter summarizes the project background, relevance, and main contributions.

The introduction concentrates on the big issues — the main questions that the project sets out to answer. We briefly present conclusions and possible future improvements for the Multi Vendor Earning Blog Site with AI.

1.1 What is the Blog Website?

The Blog Website is a full-featured multi-vendor earning platform built as a Final Year Project at Lahore Leads University. Writers from anywhere can register, publish articles across curated categories, and earn revenue based on views — similar to Medium or Substack, engineered for Pakistani university students on a modern Node.js and MongoDB stack.

Unlike a traditional single-author blog, this platform supports unlimited simultaneous writers. Each writer has their own Author Panel with Blog Analytics, Earnings Dashboard, Withdraw Funds, and Your Posts management. The platform has two distinct interfaces: the Author Panel for writers and the Admin Panel for administrators.

1.2 The Multi-Vendor Concept

Multi-vendor means many independent content creators operate simultaneously on the same platform, each acting as an independent vendor of content. Any registered user can become a content creator, managing their own articles independently, earning based on individual content performance. The platform scales naturally as more writers join — no bottleneck at a central editorial team.

1.3 Project Goals and Objectives

The Blog Website was conceived with six strategic goals, each mapping to a specific technical implementation decision:

Goal 1: Real-World Earning Platform —— Create a production-ready web app with RPM-based content monetization, view tracking, and withdrawal request management.

Goal 2: Enterprise-Grade Authentication — JWT with refresh tokens — same pattern used by Google/Facebook — for secure, stateless, scalable authentication.

Goal 3: Role-Based Access Control —— Clearly defined permission boundaries between Admin (full control) and Writer (own content only) roles.

Goal 4: AI Content Safety —— Third-party moderation API integration screening all article content before publication to prevent harmful uploads.

Goal 5: Full-Stack MVC Architecture — Complete codebase using Model-View-Controller
— industry-standard separation of concerns in web applications.

Goal 6: Security by Design —— Multiple overlapping layers: CSP headers, JWT, bcrypt hashing, input validation, environment variable security.

1.4 Technology Stack Overview

Every technology in the stack was chosen deliberately for specific reasons:

Technology	Category	Why Chosen
Node.js	Runtime	Non-blocking I/O for high-concurrency reads. Single JS language across full stack.
Express.js	Framework	Minimal, unopinionated — ideal for rapid iteration without rigid structure.
MongoDB	Database	Schema-flexible NoSQL ideal for evolving blog data models. Native JSON.
Mongoose	ODM	Schema validation, middleware hooks, type casting on top of MongoDB.
EJS	Templates	Dynamic HTML server-side rendering without complex build step.
JWT	Authentication	Stateless tokens eliminate server-side session state — horizontally scalable.
bcrypt	Password Hashing	Adaptive hashing algorithm, intentionally slow — industry gold standard.
Tailwind CSS	Styling	Utility-first CSS — no context-switching between HTML and CSS files.
Helmet / CSP	Security	Single middleware setting all recommended HTTP security headers.
dotenv	Config	Load .env files into process.env — secrets never hardcoded in source.

Table 1-1 Technology Stack Overview

1.5 Scope of Project

The scope encompasses: full-stack web application development, secure JWT authentication with bcrypt, RBAC, AI content moderation, configurable RPM earning system, withdrawal request management, contact messaging, platform announcements, CSP security hardening, responsive EJS frontend, and complete UML/ER documentation.

Not included (future work): native mobile apps, automated payment gateway processing, email notification system, full-text search, or real-time live comments.

1.6 Report Structure

Chapter 2 reviews existing literature. Chapter 3 specifies requirements. Chapter 4 presents system design and UML diagrams. Chapter 5 documents implementation with actual project screenshots in sequence. Chapter 6 covers testing results. Chapter 7 concludes and identifies future work.

Chapter 2

2 Literature Review

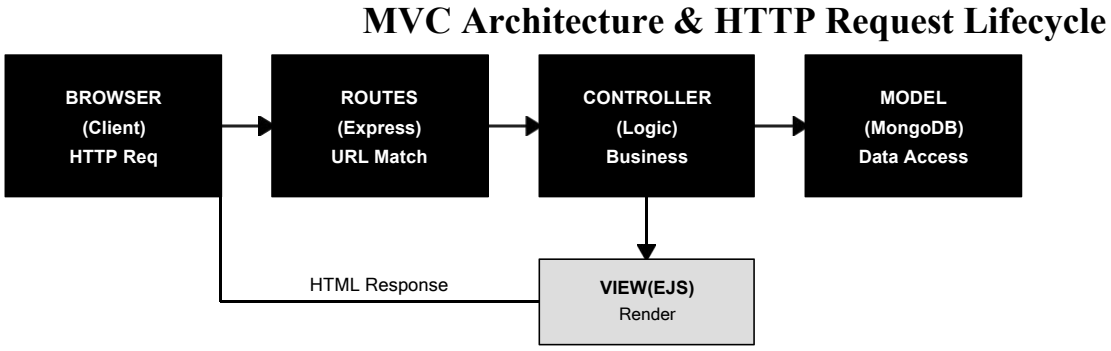


Figure 2-1 MVC Architecture and HTTP Request Lifecycle

The background section sets the project into context by relating it to existing published work which was read at the start of the project. The background section can be included as part of the introduction but is usually better as a separate chapter, especially if the project involved significant amount of prior research.

The multi-vendor blogging concept is well established in platforms such as Medium (2012), Substack (2017), and WordPress.com. Each addresses content monetization differently. Medium uses subscription-based partner programmes. Substack relies on direct newsletter subscriptions. The Blog Website adopts a view-based RPM earning model similar to Google AdSense revenue sharing, most accessible for new Pakistani content creators.

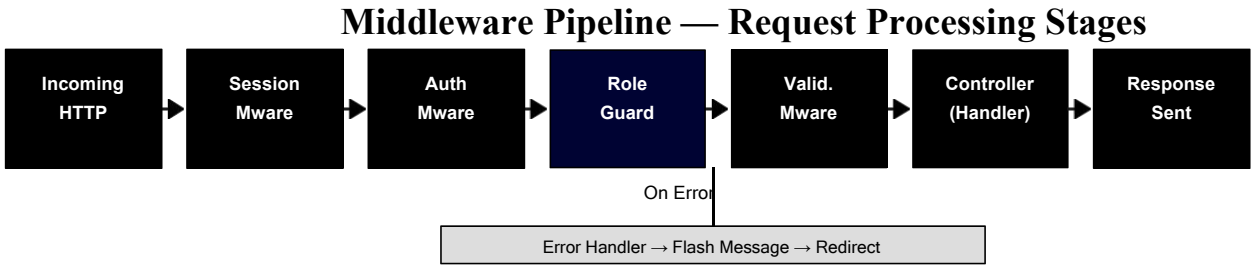


Figure 2-2 Middleware Pipeline — Request Processing Stages

Content moderation in user-generated content (UGC) platforms has been extensively studied. Automated moderation using machine learning classifiers has become standard practice. Platforms like YouTube, Facebook, and Twitter employ AI-based pre-publication screening. The integration of a moderation API in this project follows established best practices documented by major cloud providers.

JWT-based authentication is documented in RFC 7519 and is the industry standard for stateless web authentication. The dual-token approach (short-lived access token plus long-lived refresh token) is the OWASP-recommended pattern for balancing security with usability.

The MVC architectural pattern, first described by Trygve Reenskaug in 1979 at Xerox PARC, remains the dominant pattern for web application structure. Express.js with EJS server-side rendering follows this pattern natively.

WARNING: Avoid plagiarism

If you take another person's work as your own and do not cite your sources of information you are being dishonest; in other words you are cheating. When referring to other pieces of work, cite the sources where they are referred to or used, rather than just listing them at the end.

Chapter 3

3 Requirements Specification

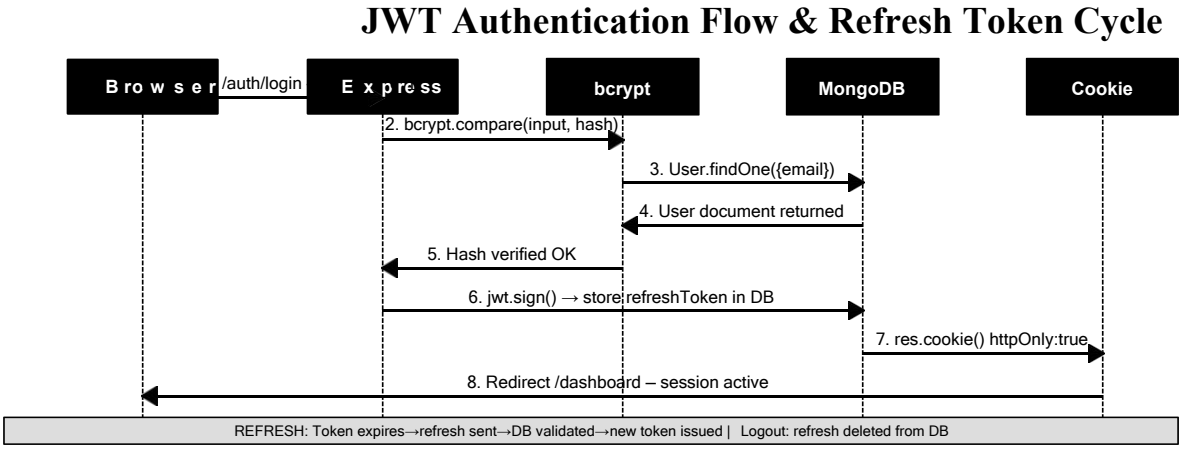


Figure 3-1 JWT Authentication Flow & Refresh Token Cycle

The Non-functional and Functional Requirements are categorized into various groups based on relations and objectives of requirements. Each requirement is assigned an ID: FR-GroupIndex-RequirementIndex for Functional and NR-GroupIndex-RequirementIndex for Non-Functional Requirements. Priority scale 1-3: Priority 1 must be fully implemented; Priority 2 secondary; Priority 3 may be deferred.

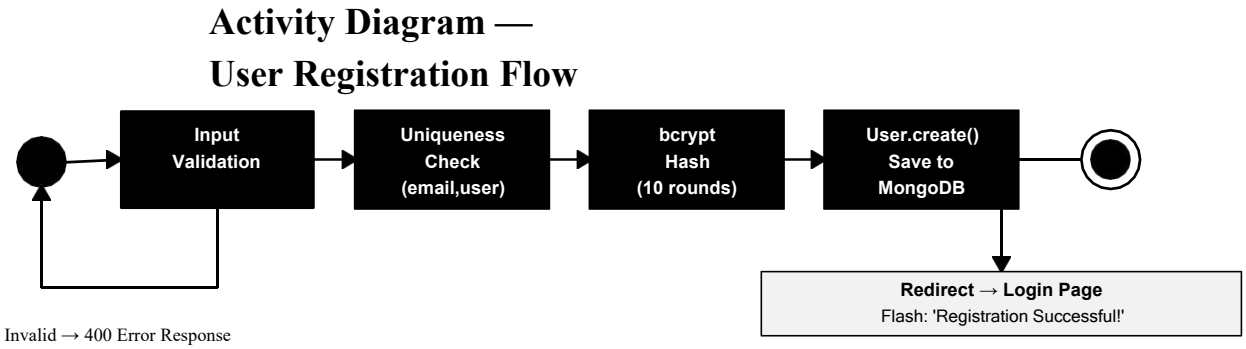


Figure 3-2 Activity Diagram — User Registration Flow

3.1 Functional Requirements

3.1.1 Authentication and Content Requirements

ID	Feature	Description	Pri
FR-01-001	User Registration	Register with email, password, username; bcrypt hashing applied	1

ID	Feature	Description	Pri
FR-01-002	User Login	Login with JWT tokens stored in HTTP-only cookies	1
FR-01-003	User Logout	Session destroy, clear cookies, revoke refresh token in DB	1
FR-01-004	Password Change	Verify current password before accepting new bcrypt hash	1
FR-02-001	Article Creation	Create/publish articles with category, topic, image, tags	1
FR-02-002	AI Moderation	All article content passes AI moderation API before DB save	1
FR-02-003	Article Management	Writers view, edit and delete their own articles via Your Posts	1
FR-03-001	Admin Dashboard	Protected panel with blogs, views, likes, shares, comments stats	1
FR-03-002	Category Management	Create, edit, and assign topics to categories with RPM groups	1
FR-03-003	RPM Rate Config	Set earning rate per 1000 views per RPM level (Low/Normal/Medium/High)	1
FR-04-001	Earning Tracking	Track views, calculate earnings, update writer balance	1
FR-04-002	Writer Dashboard	View Blog Analytics: total/monthly/today blogs, views, likes, shares	1
FR-04-003	Withdrawal System	Submit withdrawal requests; admin approve/decline management	1
FR-05-001	Contact Messages	Writers send contact messages; admin views/manages them	2
FR-05-002	Announcements	Admin posts platform announcements visible to all writers	2

Table 3-1 Functional Requirements

3.2 Non-Functional Requirements

ID	Category	Description	Pri
NR-01-001	Security	Passwords hashed with bcrypt 10 salt rounds before storage	1
NR-01-002	Security	All API routes protected with JWT middleware verification	1
NR-02-001	Performance	Page load time must not exceed 3 seconds on standard connection	2
NR-02-002	Scalability	Stateless JWT supports horizontal server scaling without shared state	2
NR-03-001	Usability	Responsive mobile-friendly interface with Tailwind CSS utility classes	2
NR-04-001	Maintainability	MVC pattern with strict separation of concerns throughout codebase	1
NR-04-002	Maintainability	Single responsibility principle per route, controller, and model file	1

Table 3-2 Non-Functional Requirements

Chapter 4

4 Project Design

4.1 Use Case Diagrams

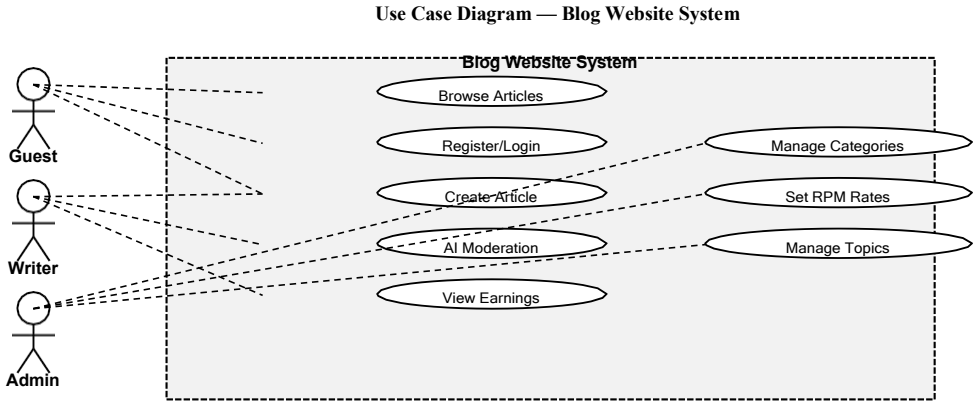


Figure 4-1 Use Case Diagram — Blog Website System

The use case diagram illustrates interactions between actors (Guest, Registered Writer, Admin) and the system. Primary use cases include Register/Login, Create Article, Browse Articles, Manage Categories/Topics, View Earnings Dashboard, and AI Content Moderation. The system boundary encompasses all functionality served by the Express.js backend.

4.2 State Diagrams

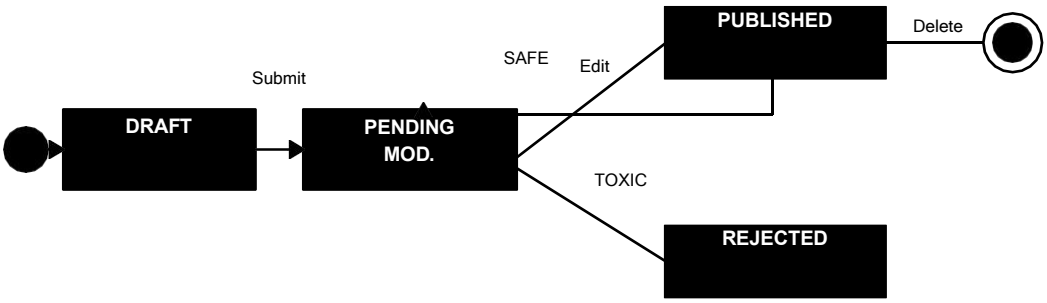


Figure 4-2 Article State Diagram — Lifecycle

The article state diagram defines the lifecycle: Draft → Pending Moderation → Published (if SAFE) or Rejected (if TOXIC/HARMFUL). A published article can be edited (returning to Pending Moderation) or deleted by the author. End state shown with double-circle notation for article deletion.

4.3 Sequence Diagrams

Key sequence diagrams: (1) Login Flow — Browser sends credentials to Express, which queries MongoDB, compares bcrypt hash, generates JWT tokens, sets HTTP-only cookies. (2) Article Creation Flow — Writer submits form, JWT validated, moderation API called, on SAFE result saves to MongoDB. (3) Token Refresh Flow — client sends refresh token, server validates against DB, issues new access token.

4.4 Activity Diagrams

Activity diagrams capture two critical workflows: user registration (input validation → uniqueness check → bcrypt hash → account creation → redirect to login) and article publication (form submission → JWT check → validation → AI moderation → DB save → success redirect).

4.5 Collaborative Diagrams

Collaborative diagrams show object interactions in the MVC pattern. Controller objects (authController, articleController, adminController) collaborate with Model objects (User, Category, Topic, Article) and View objects (EJS templates). Controllers never query MongoDB directly — they delegate to Models.

4.6 Class Diagram

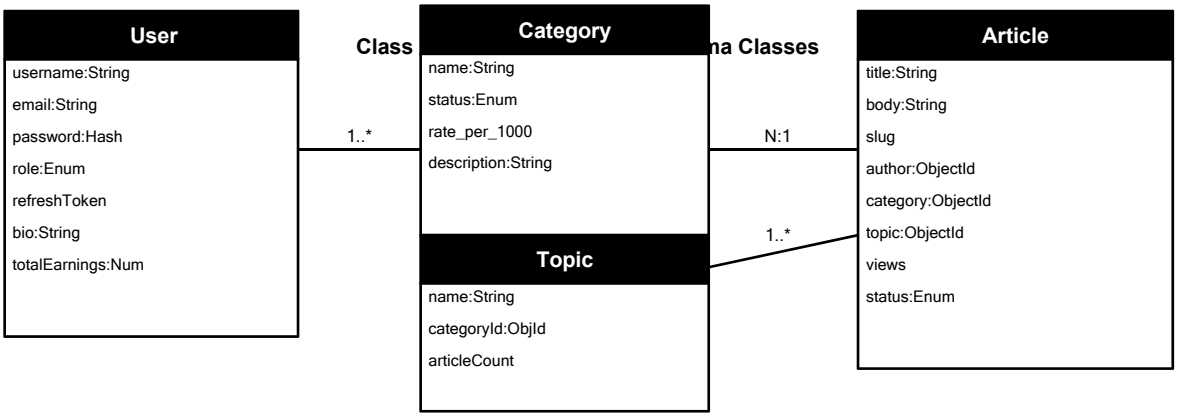


Figure 4-3 Class Diagram — Mongoose Schema Classes

The class diagram shows four main Mongoose schema classes. User contains authentication and earnings fields. Category holds name, status, and RPM rate. Topic references its parent Category via ObjectId. Article references User (author), Category, and Topic. Mongoose's .populate() performs JOIN-equivalent operations between collections.

4.7 UML Diagram

The UML component diagram shows: Client (Browser), Web Server (Express.js on Node.js), Database (MongoDB Atlas), External Services (AI Moderation API). The

middleware pipeline: Session Middleware → Auth Middleware → Role Guard → Validation Middleware → Controller → Response. Any layer failure terminates the request with the appropriate HTTP status code.

4.8 ER Diagram

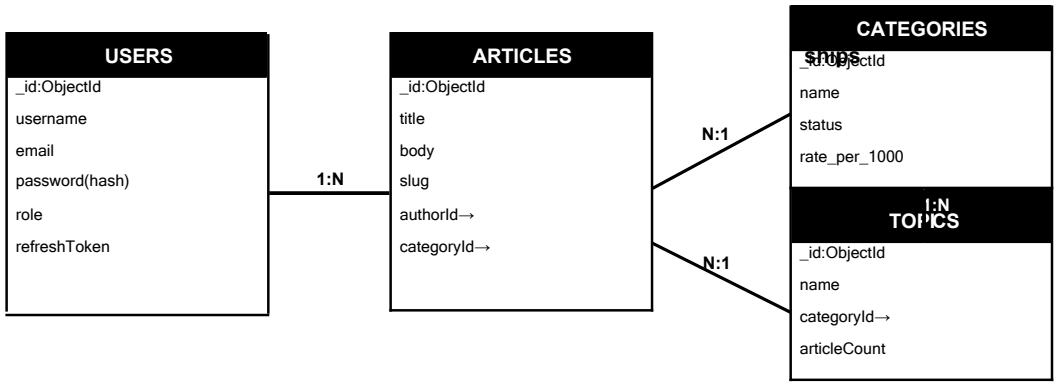


Figure 4-4 ER Diagram — MongoDB Collections & Relationships

The ER diagram defines four MongoDB collections and their relationships. Users has 1:N with Articles. Categories has 1:N with Topics. Articles reference Category and Topic (N:1 each). All foreign key references use MongoDB ObjectId with Mongoose population for JOIN-equivalent queries.

Chapter 5

5 Implementation

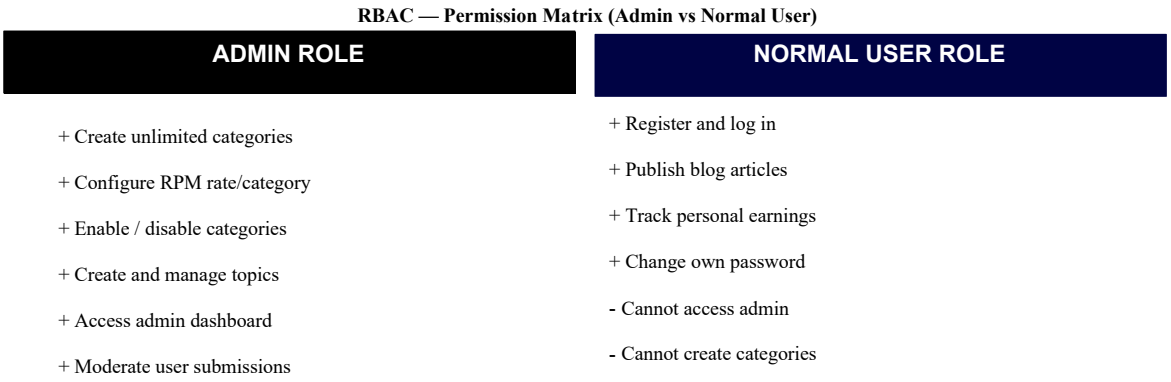


Figure 5-1 RBAC — Permission Matrix (Admin vs Normal User)

A modular, production-ready prototype was developed following the MVC pattern. The system has no separate API server and frontend client — both are served by the same Express application (monolithic architecture), simplifying deployment for a university project.

5.1 Tools and Technologies

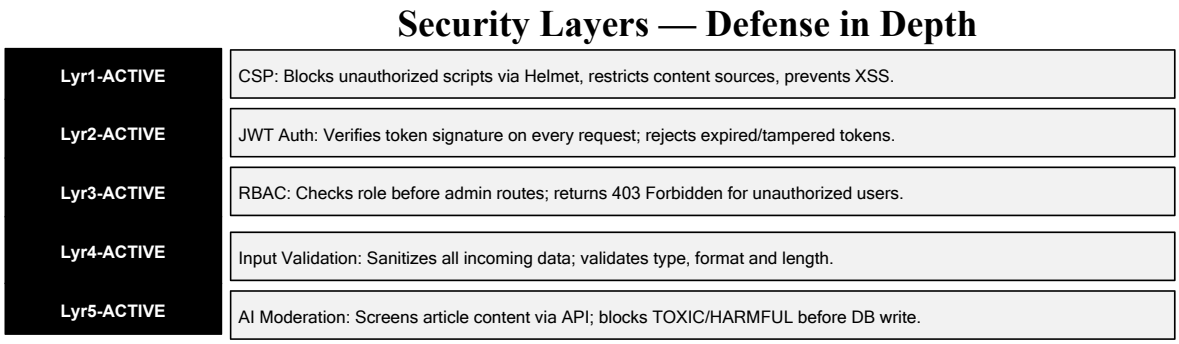


Figure 5-2 Security Layers — Defense in Depth (CSP, JWT, RBAC, Validation,
AI)

Development environment: Node.js v18+ LTS, npm v9+, MongoDB 6.0+/Atlas cloud, Git for version control, VS Code as primary editor. The project follows the Open/Closed Principle: new features added via new controller/route/model files without modifying existing ones. Full npm package list is in Appendix A.

5.2 Design Description

The following modules constitute the developed product. Each module is documented with its actual working screenshots in sequence, followed by technical implementation details.

5.2.1 New Post Creation Module

This is the first module a writer uses after logging in. The Create Post workflow is the core feature of the entire platform — it is where AI content moderation is triggered and where writers submit their articles for publication.

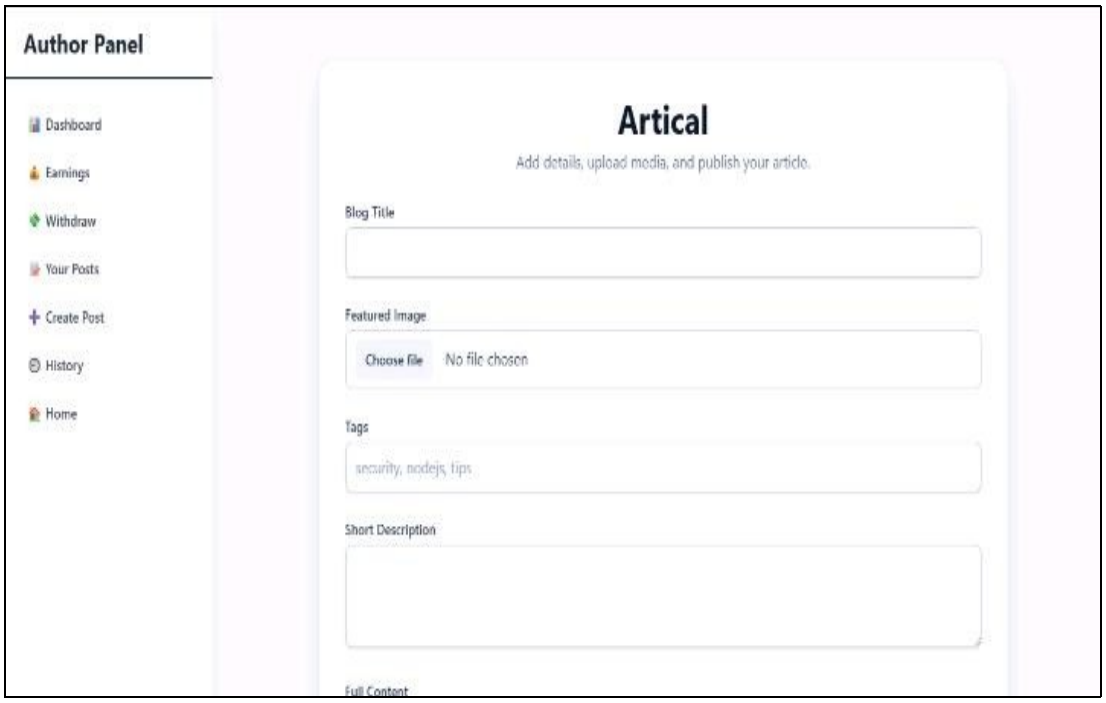


Figure 5-3 Create Article Form (New Post) — Author Panel

Working: The writer navigates to 'Create Post' in the Author Panel sidebar. The form collects Blog Title (10-200 characters), Featured Image upload (optional), Tags (comma-separated keywords like 'security, nodejs, tips'), Short Description (article summary), and Full Content (rich text body, minimum 100 characters). On clicking Publish, the system sends the title and body content to the AI Moderation API. If the API returns SAFE, the article is saved to MongoDB and the writer sees a green success message. If the AP I returns TOXIC or POTENTIALLY_HARMFUL, the article is blocked and the writer receives an error message explaining that the content must be revised before resubmission. The form also supports featured image upload using Multer middleware for binary file handling.

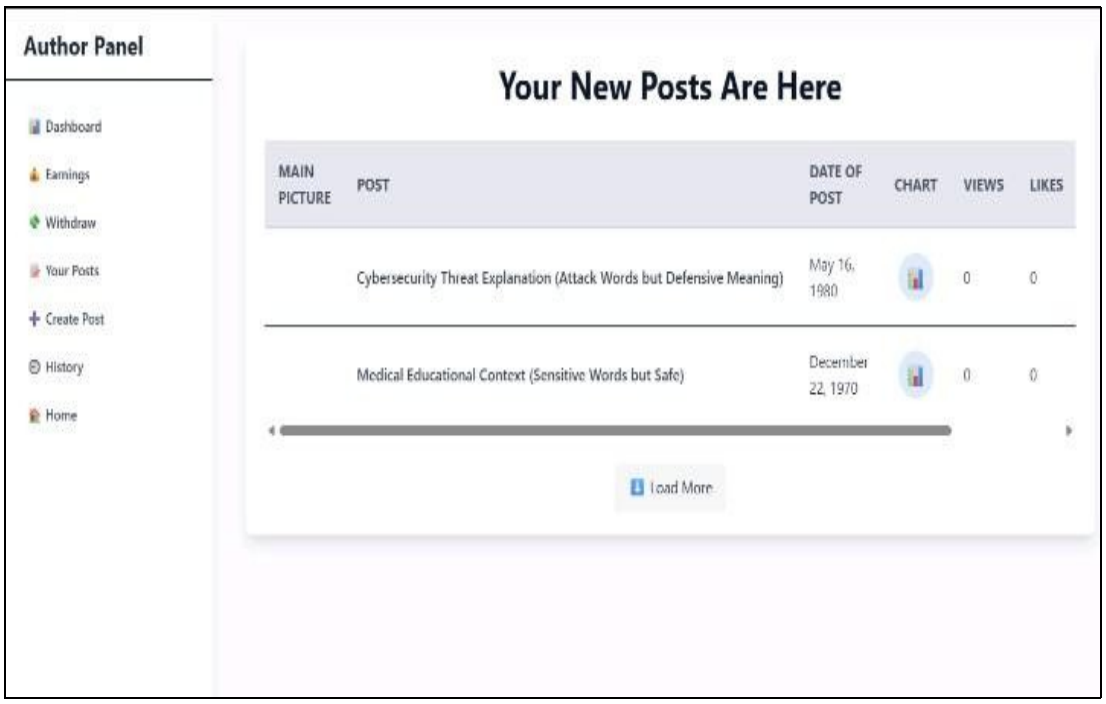


Figure 5-4 Your Posts Page — Article List with Views & Likes

Working: After publishing, the writer's articles appear in the 'Your Posts' section. The table displays: Main Picture (thumbnail), Post Title, Date of Post, a Chart icon (links to analytics), Views count, and Likes count. Each row represents one published article. Writers can scroll through their article portfolio and click 'Load More' for pagination. The chart icon links to the per-article performance analytics. Articles published with test content (like 'Cybersecurity Threat Explanation' and 'Medical Educational Context') both passed AI moderation as they are educational, not harmful, demonstrating that the AI correctly classifies context-aware content.

5.2.2 Blog Analytics & Author Dashboard Module

The Blog Analytics Dashboard gives writers a comprehensive overview of their platform performance. This module aggregates statistics from the articles collection and presents them in card-based and chart-based views.



Figure 5-5 Blog Analytics Dashboard — Author Panel

Working: The dashboard displays five metric cards: Blogs (Total Blogs, This Month, Today), Views (Total Views, This Month, Today), Likes (Total Likes, This Month, Today), Share (Total Shares, This Month, Today), and Comments (Total Comments, This Month, Today). Each card updates in real time as readers interact with the writer's articles. Below the cards, a 'Performance Trend' chart (Last 7 Days dropdown) visualizes engagement over time. The sidebar shows the complete Author Panel navigation: Dashboard, Earnings, Withdraw, Your Posts, Create Post, History, Home. This module is built using EJS server-side rendering — the controller fetches aggregated MongoDB data and passes it to the EJS template for display.

5.2.3 Earnings & Withdrawal Module

The Earnings module is the economic heart of the platform. It tracks how much each writer has earned based on article views and the RPM (Rate Per Mille) assigned to their article's category. This module also handles the withdrawal request workflow.

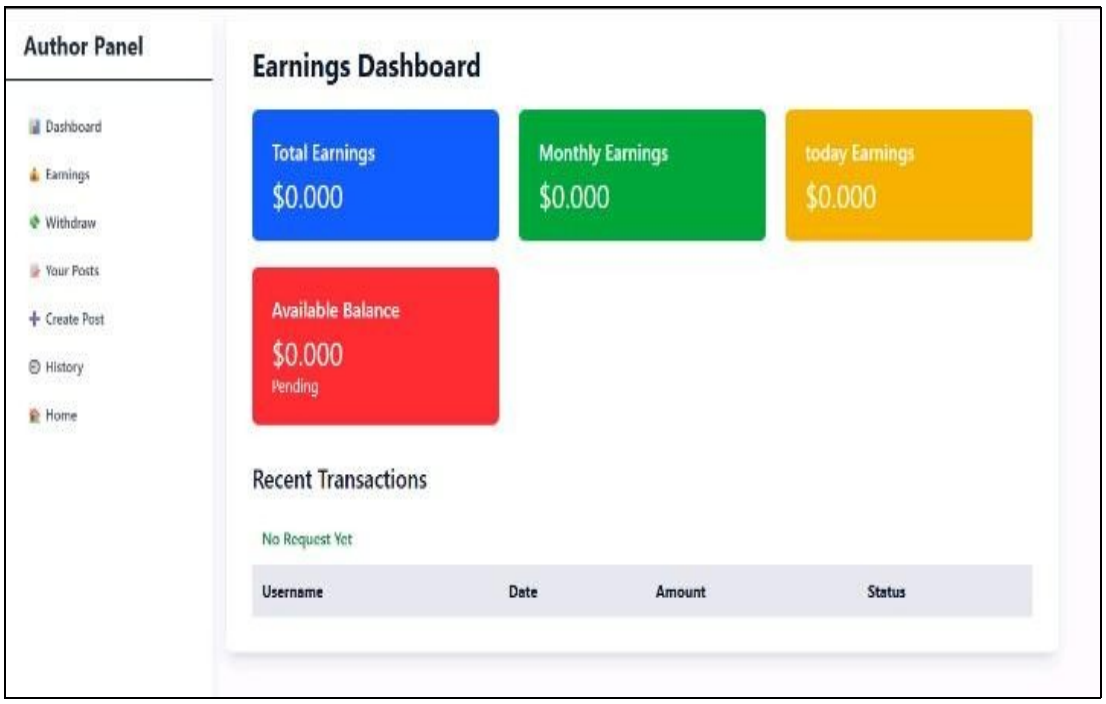


Figure 5-6 Earnings Dashboard — Author Panel

Working: The Earnings Dashboard displays four metric cards: Total Earnings (cumulative balance in USD, shown in blue), Monthly Earnings (this month's total, shown in green), Today's Earnings (today's total, shown in yellow/gold), and Available Balance with Pending indicator (shown in red). The formula applied is: Earnings = (Article Views ÷ 1000) × RPM Rate. For example, an article with 1,000 views in a Medium RPM category ($10) earns $10.00. Below the cards, the 'Recent Transactions' section shows a table with Username, Date, Amount, and Status columns. When no withdrawal has been requested, it shows 'No Request Yet'. All earnings data is fetched from the MongoDB articles collection and aggregated in the earnings controller.

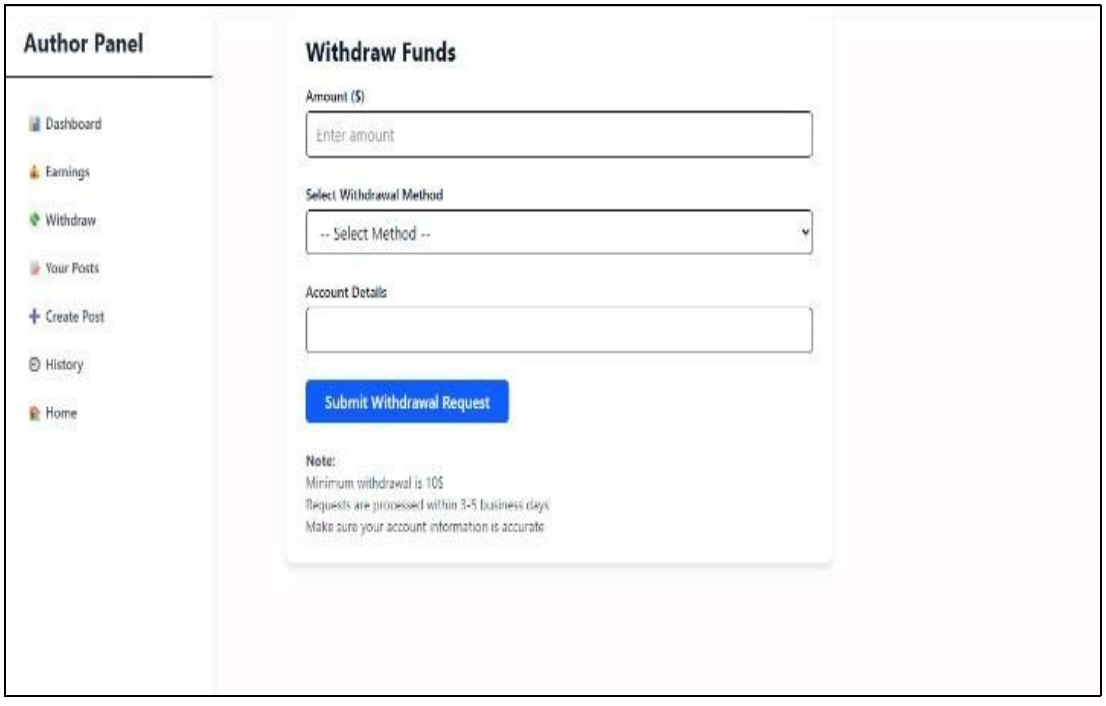


Figure 5-7 Withdraw Funds Form — Author Panel

Working: The Withdraw Funds form allows writers to request payment of their accumulated earnings. Fields: Amount ($) — the writer enters the amount they wish to withdraw (minimum $10 as noted at the bottom). Select Withdrawal Method — a dropdown to choose the payment method (bank transfer, JazzCash, Easypaisa etc.). Account Details — a text field for the writer's account number or payment details. On clicking 'Submit Withdrawal Request', the system creates a withdrawal document in MongoDB with status 'pending'. The admin then reviews this request in the Admin Panel's Withdrawal Requests section and can Approve or Decline it. Notes shown on the form: minimum withdrawal is $10, requests processed within 3-5 business days, account information must be accurate.

5.2.4 Admin Dashboard Module

The Admin Panel is a protected interface accessible only to users with role='admin'. The adminGuard middleware enforces this restriction on every admin route, returning 403 Forbidden to any non-admin user. The Admin Dashboard provides a platform-wide overview of all activity.

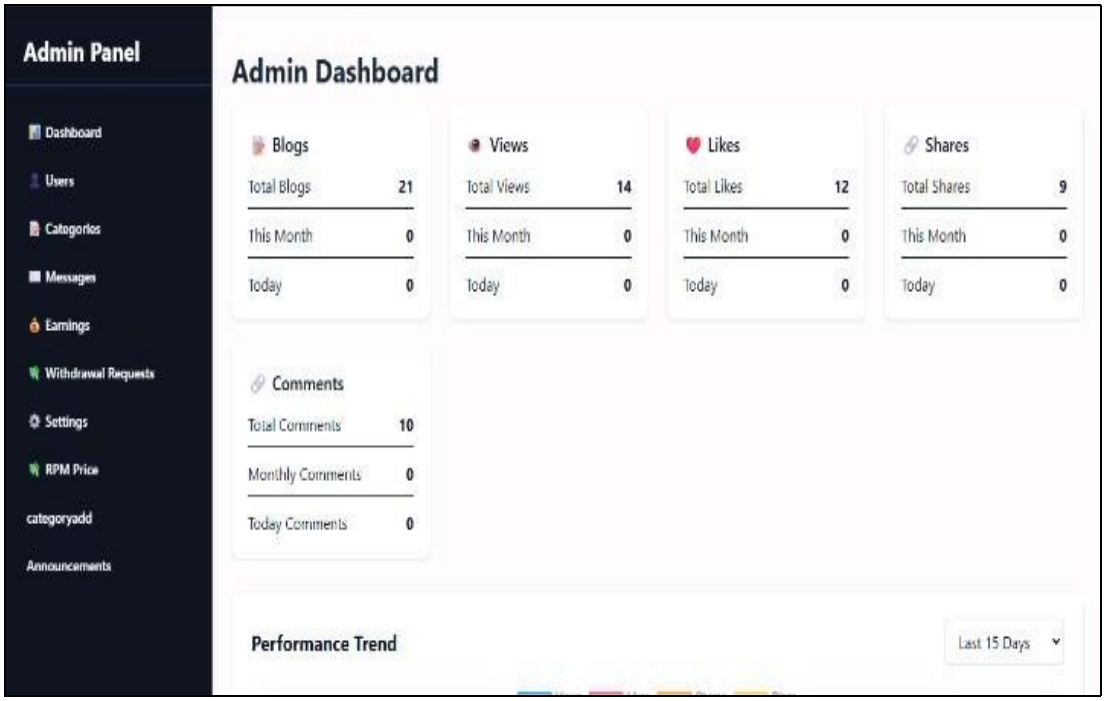


Figure 5-8 Admin Dashboard — Overview Panel

Working: The Admin Dashboard displays five platform-wide metric cards: Blogs (Total: 21, This Month: 0, Today: 0), Views (Total: 14, This Month: 0, Today: 0), Likes (Total: 12, This Month: 0,
Today: 0), Shares (Total: 9, This Month: 0, Today: 0), and Comments (Total: 10, Monthly: 0, Today: 0). The dark sidebar shows the complete Admin Panel navigation: Dashboard, Users, Categories, Messages, Earnings, Withdrawal Requests, Settings, RPM Price, categoryadd, Announcements. Below the metric cards, a 'Performance Trend' chart (Last 15

Days) visualizes platform-wide activity over time. All data is fetched from MongoDB using aggregation pipelines in the adminController, which groups documents by day and sums up the relevant metrics.

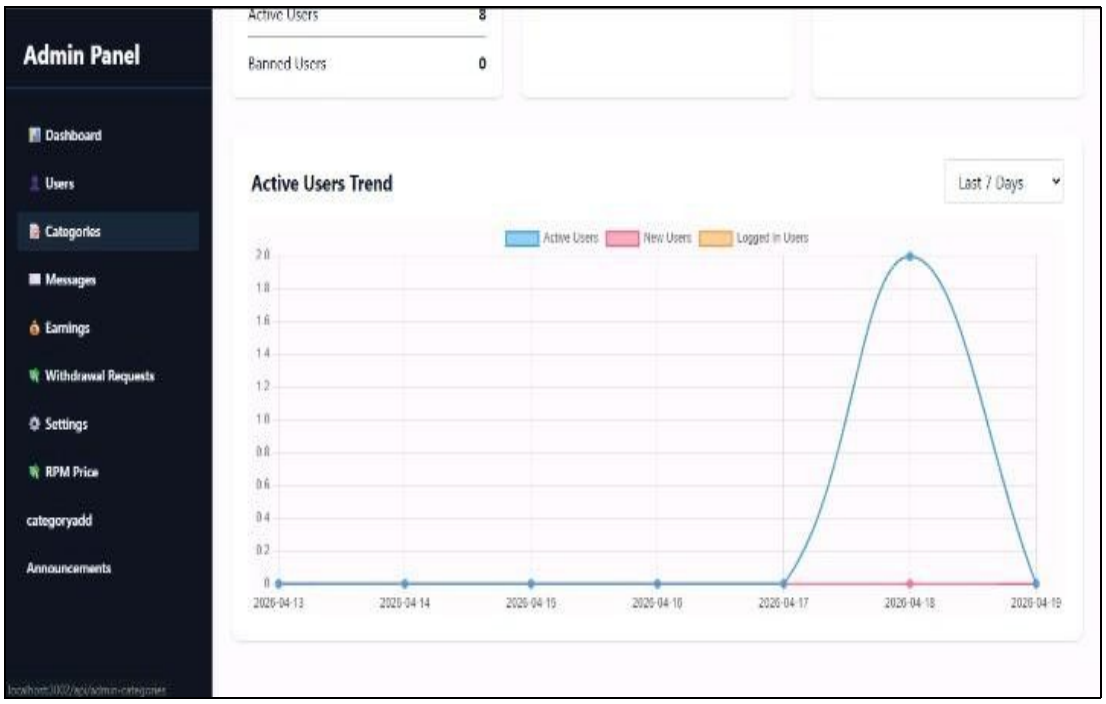


Figure 5-9 Admin Dashboard — Active Users Trend Chart

Working: The Active Users Trend section of the Admin Dashboard shows a line chart plotting user activity over the last 7 days (2026-04-13 to 2026-04-19). The chart legend shows three series: Active Users (blue line), New Users (pink line), and Logged-in Users (yellow/orange line). The peak of 2.0 active users was recorded on 2026-04-18. This chart helps administrators understand platform usage patterns and identify growth trends. The data is retrieved from the sessions collection in MongoDB, grouped by date using a MongoDB aggregation pipeline with $group, $sum, and $dateToString operators. The chart is rendered using Chart.js on the client side, with data passed from the server as JSON embedded in the EJS template's data attributes.

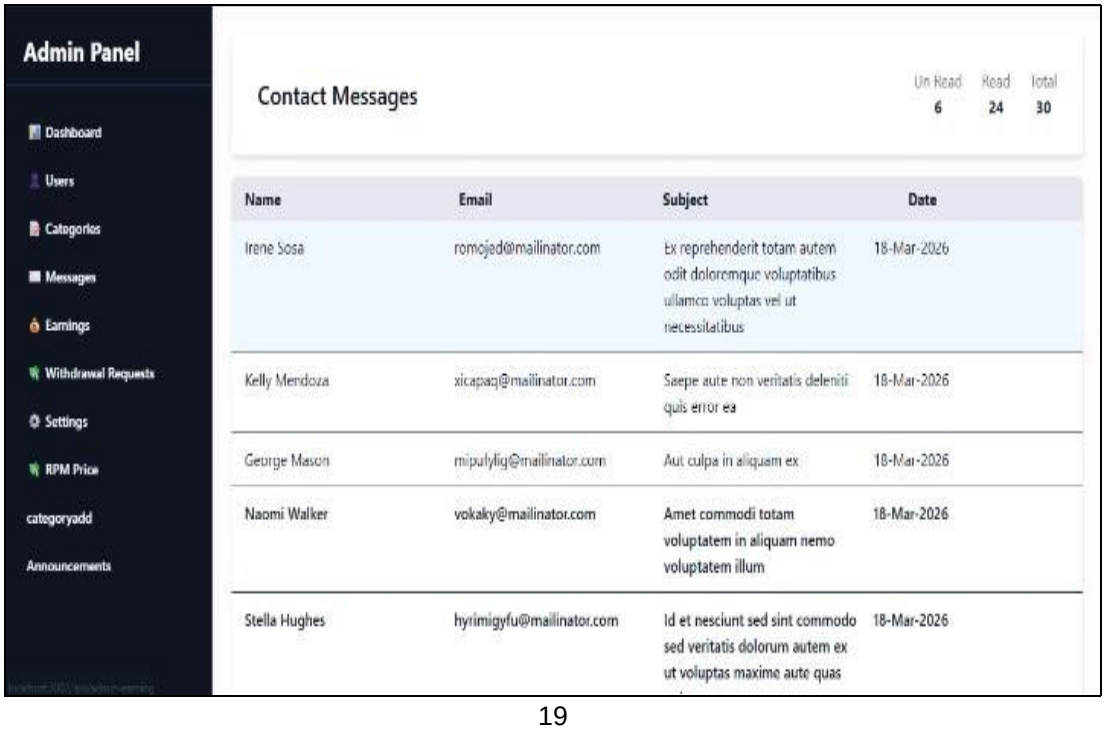


Figure 5-11 Contact Messages — Admin Panel

Working: The Contact Messages page shows all messages submitted through the platform's contact form. Stats displayed: Unread: 6, Read: 24, Total: 30. The table shows Name, Email, Subject, and Date for each message. Admin can click each row to read the full message and mark it as read. This module uses a separate 'messages' MongoDB collection. The adminController fetches all messages sorted by date descending and categorizes them by read status. The page enables admins to respond to writer queries, platform issues, or partnership requests. The page shown contains test data from a randomized seed (Mailinator emails are test email addresses).

5.2.5 Admin Earnings & RPM Management Module

The Admin Earnings module gives administrators a platform-wide view of all financial activity. The RPM Management module allows administrators to configure the earning rates that determine how much writers earn per 1000 article views.



Figure 5-12 Earnings Overview — Admin Panel with Trend Chart

Working: The Admin Earnings Overview displays four key financial metrics: Total Earnings ($0.038
— accumulated from test article views), This Month ($0.000), Today ($0.000), and Pending Withdrawals ($0.00). Below, an 'Earnings Trend' line chart shows earnings activity over configurable time periods: Last 7 Days (selected), Last 15 Days, Last 1 Month, Last 3 Months. The chart uses Chart.js rendered client-side with data from a MongoDB aggregation pipeline that groups earnings by date. The adminController calculates total earnings by summing all writer balances from the users collection and pending withdrawals from the withdrawals collection.

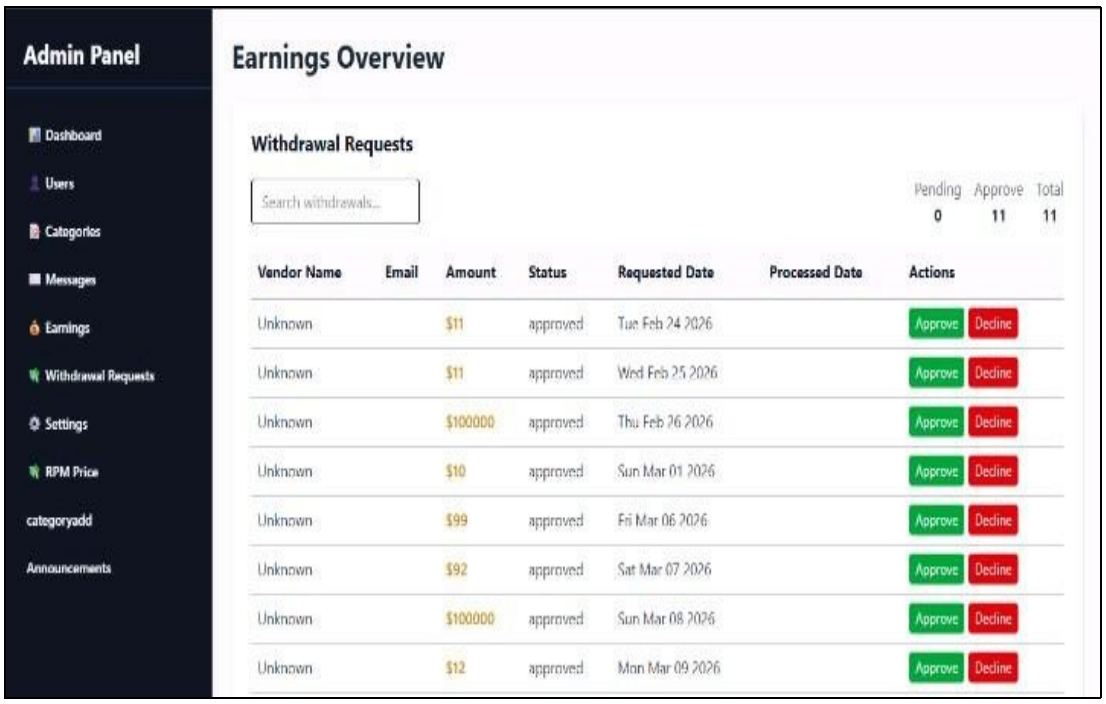


Figure 5-13 Withdrawal Requests — Admin Panel

Working: The Withdrawal Requests page shows all pending and processed withdrawal requests from writers. Stats: Pending: 0, Approved: 11, Total: 11. Each row shows: Vendor Name, Email, Amount (highlighted in yellow for large amounts), Status (approved), Requested Date, Processed Date, and Actions (green Approve button, red Decline button). The page includes a search bar for filtering withdrawals by vendor name or email. All 11 historical requests shown are approved. The admin can process new pending requests by clicking Approve (changes status to 'approved', triggers payment processing) or Decline (changes status to 'declined', returns amount to writer's available balance). The data is stored in a withdrawals MongoDB collection with status field as Enum: ['pending', 'approved', 'declined'].

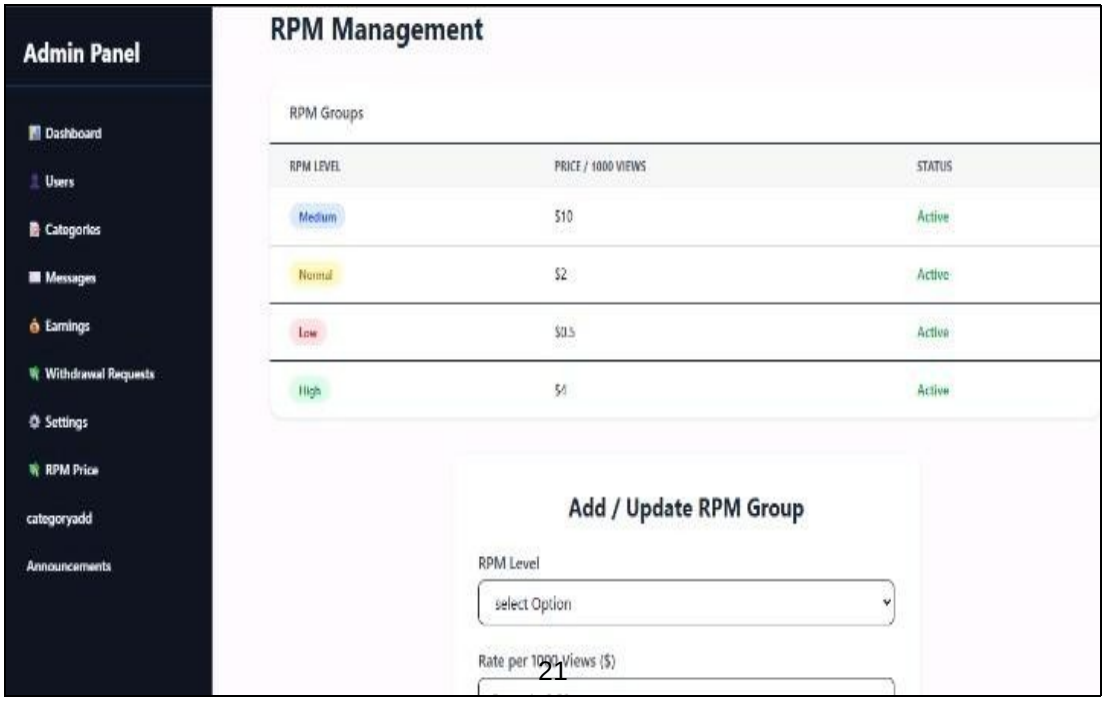


Figure 5-15 RPM Management — Rate per 1000 Views Configuration

Working: The RPM Management page allows admins to configure earning rates for different RPM levels. The RPM Groups table shows four levels currently configured: Medium ($10/1000 views, Active), Normal ($2/1000 views, Active), Low ($0.5/1000 views, Active), High ($4/1000 views, Active). Color-coded badges: Medium=blue, Normal=yellow, Low=red/pink, High=green. The 'Add/Update RPM Group' form below allows admins to select an RPM Level from a dropdown and set the Rate per 1000 Views. Changes apply immediately to all future earning calculations without server restart. When a writer publishes an article, its topic is assigned to an RPM group, and the earning formula becomes: Earnings += (articleViews / 1000) × rpmGroup.rate_per_1000.

5.2.6 Category Management & Announcement Module

The Category management module allows admins to organize the blog platform's content taxonomy. The Announcement module enables platform-wide communications from admin to all writers.

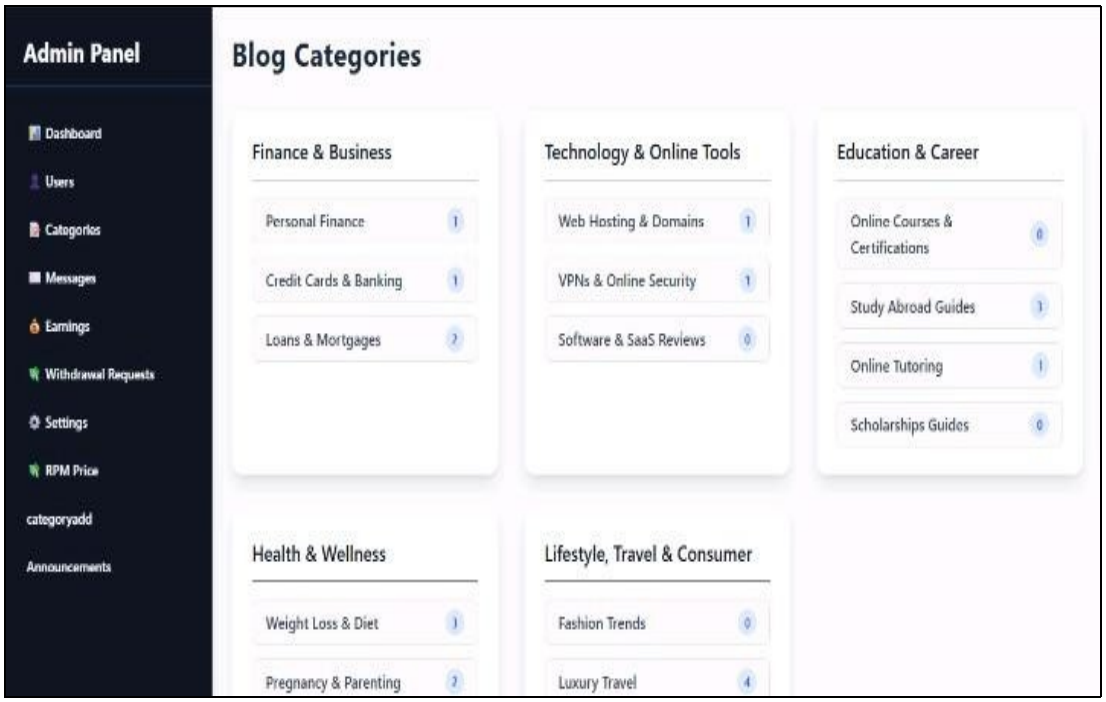


Figure 5-10 Blog Categories — Admin Panel

Working: The Blog Categories page displays all categories organized in card-based groups. Each category card shows its name and the count of articles per topic within that category. Categories shown: Finance & Business (Personal Finance: 1, Credit Cards & Banking: 1, Loans & Mortgages: 2), Technology & Online Tools (Web Hosting & Domains: 1, VPNs & Online Security: 1, Software & SaaS Reviews: 0), Education & Career (Online Courses: 0, Study Abroad Guides: 1, Online Tutoring: 1, Scholarships Guides: 0), Health & Wellness (Weight Loss & Diet: 1, Pregnancy & Parenting: 2), Lifestyle, Travel &

Consumer (Fashion Trends: 0, Luxury Travel: 4). Each topic shows its article count in a blue bubble. The category data is fetched using a MongoDB aggregation that joins categories with topics and counts associated articles.

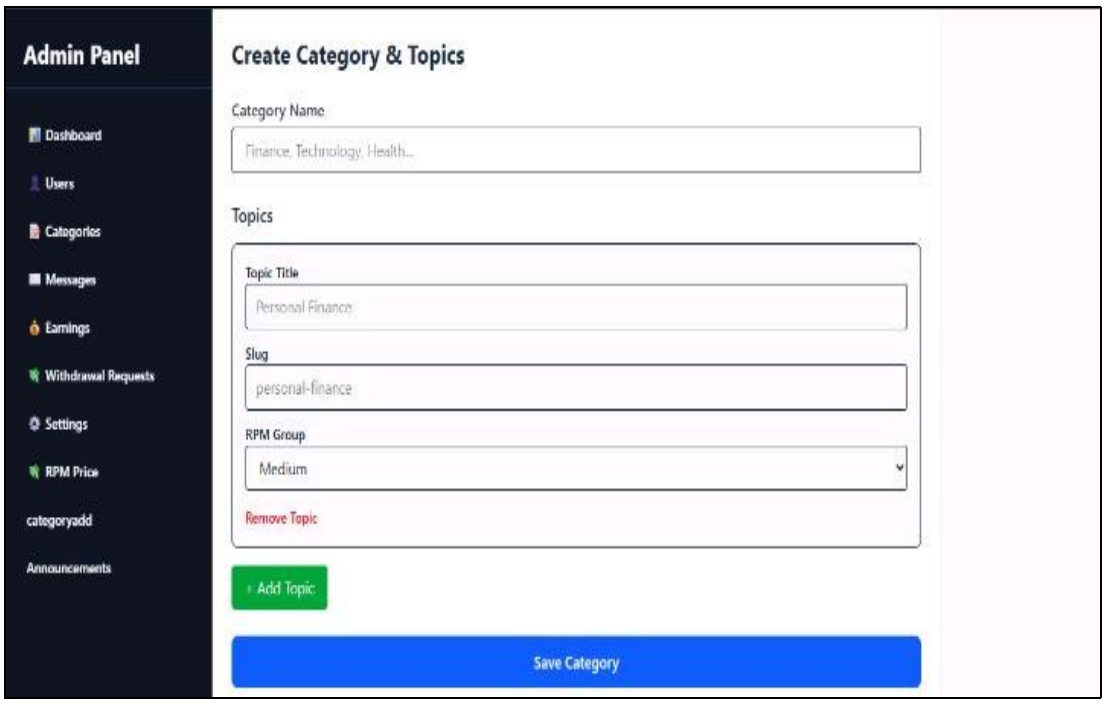


Figure 5-16 Create Category & Topics Form — Admin

Working: The 'Create Category & Topics' form allows admins to add new content categories with multiple topics in a single operation. Fields: Category Name (placeholder: Finance, Technology, Health...). Topics section shows one topic form by default with: Topic Title (placeholder: Personal Finance), Slug (auto-generated URL-friendly version: personal-finance), RPM Group (dropdown: Medium selected). The '+ Add Topic' button (green) dynamically adds more topic forms to the same category. 'Remove Topic' (red text) removes a topic. On clicking 'Save Category' (blue full-width button), all topics are saved to MongoDB with ObjectId references to the parent category. The slug field uses the slugify npm package to automatically generate URL-safe identifiers from topic titles.

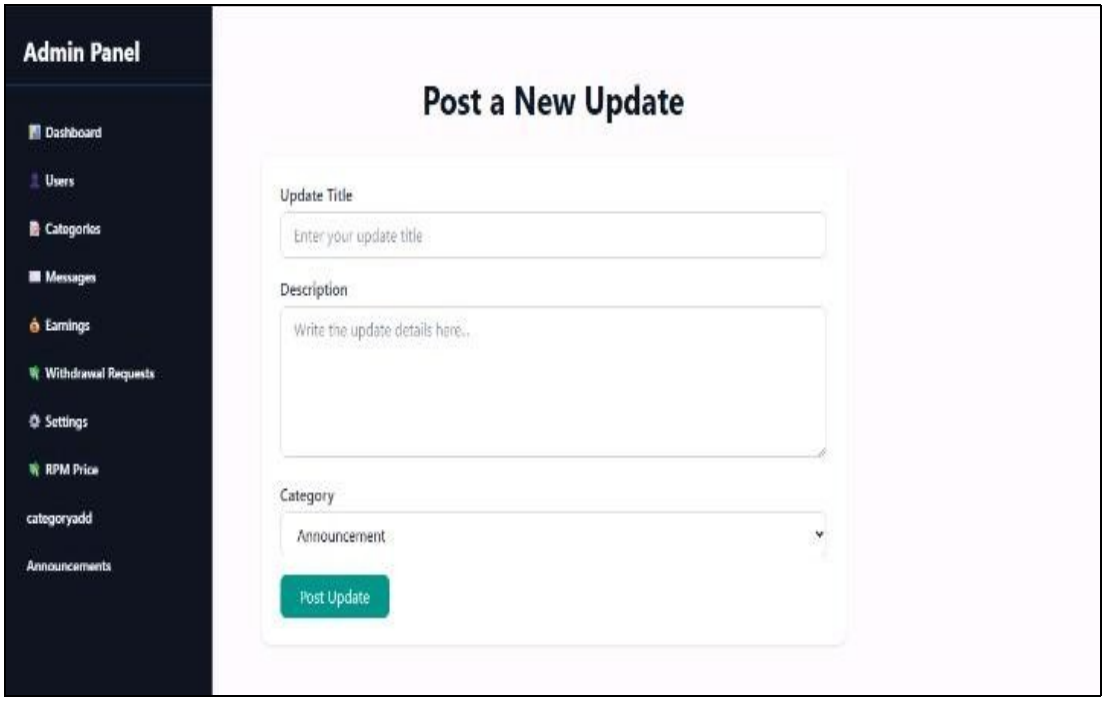


Figure 5-17 Post a New Announcement — Admin Panel

Working: The Announcements module allows admins to post platform-wide updates to all writers. Fields: Update Title (placeholder: Enter your update title), Description (large textarea: Write the update details here...), Category dropdown (Announcement selected). Clicking 'Post Update' (teal/green button) creates a new announcement document in the MongoDB announcements collection with title, description, category, timestamp, and admin author ID. Announcements are then displayed to all writers when they log in or visit their dashboard. This is used for platform policy changes, new feature announcements, maintenance windows, or earning rate changes.

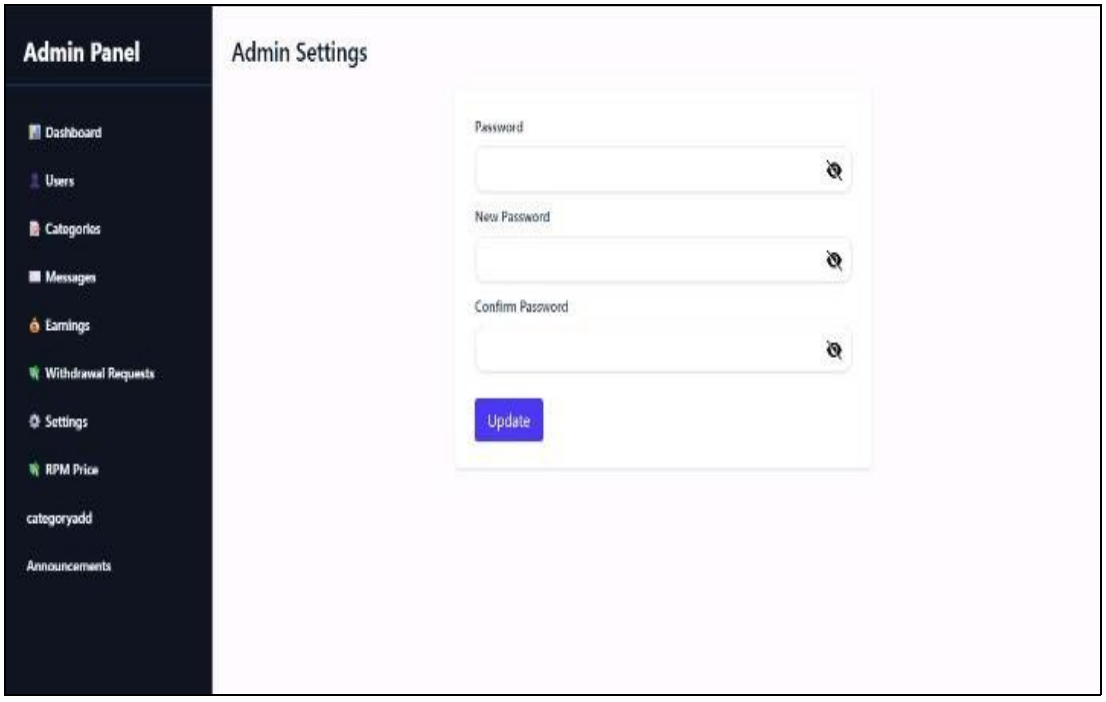


Figure 5-14 Admin Settings — Password Change

Working: The Admin Settings page provides a secure password change form for the administrator account. Fields: Password (current password — required for verification), New Password (new bcrypt hash will be stored), Confirm Password (must match New Password). The eye icon on each field toggles password visibility (show/hide). On clicking 'Update' (purple button), the system: (1) verifies the current password using bcrypt.compare(), (2) if valid, hashes the new password with bcrypt.hash(newPassword, 10), (3) updates the admin user document in MongoDB, (4) returns a success message. If the current password is wrong, a 400 error is returned. This is the same password change flow used by writer accounts but rendered in the Admin Panel context.

RPM Level	Rate/1K Views	Example: 5K Views	Example: 100K Views
High	$4.00	$20.00	$400.00
Medium	$10.00	$50.00	$1,000.00
Normal	$2.00	$10.00	$200.00

Low	$0.50	$2.50	$50.00

Table 5-1 RPM Earning Rate Configuration (actual values from RPM Management screen)

Status	Writer Can Select	Existing Articles	Earning Continues
active	YES	Visible and accessible to all readers	YES —— RPM rate applied to all new views
inactive	NO —— hidden	Remain published and readable	YES — existing articles keep earning

Table 5-2 Category Status Control — Visibility and Earning Rules

Chapter 6

6 Testing

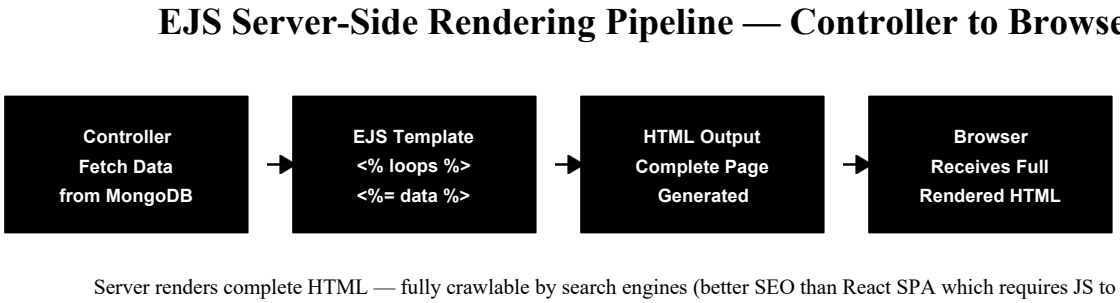


Figure 6-1 EJS Server-Side Rendering Pipeline — Controller to Browser

We focused on thorough testing throughout the design and implementation phase. Testing was performed at unit level (individual modules) and integration level (full system). Both functional and security aspects were tested systematically.

6.1 Unit Testing

Each module was tested while being developed to confirm adherence to requirements. The authentication module was tested for: correct password hashing, JWT generation and verification, refresh token lifecycle, and smart redirect logic. The content moderation module was tested with sample SAFE, POTENTIALLY_HARMFUL, and TOXIC content strings. The admin module was tested for role enforcement ensuring 403 Forbidden on unauthorized access attempts.

6.2 Function Testing

After integrating the system, testing was done end-to-end simulating real user workflows. The following test cases were systematically executed and verified:

6.2.1 Testing Requirements — Authentication, Content, Admin, and Earning

Test ID	Test Case	Expected Result	Status
TC-001	Register with valid data	Account created, redirect to login with success flash	PASS
TC-002	Register with duplicate email	400 error: email already exists in database	PASS
TC-003	Register with password < 8 chars	400 error: password minimum 8 characters	PASS
TC-004	Login with correct credentials	JWT cookies set, redirect to Author Dashboard	PASS
TC-005	Login with wrong password	401: Invalid credentials (generic message)	PASS
TC-006	Access /dashboard without login	Redirect to /auth/login with flash message	PASS

Test ID	Test Case	Expected Result	Status
TC-007	Normal user access /admin	403 Forbidden returned by adminGuard middleware	PASS
TC-008	Submit educational article (SAFE)	Article saved to DB, success flash shown to writer	PASS
TC-009	Submit harmful article (TOXIC)	DB write blocked, error flash shown to writer	PASS
TC-010	Admin create category with topics	Category and topics created, appear in admin list	PASS
TC-011	Admin set RPM rate (Medium=$10)	Rate saved, applies to future earning calculations	PASS
TC-012	Writer views Blog Analytics	Dashboard shows blogs, views, likes, shares, comments	PASS
TC-013	Writer submits withdrawal request	Withdrawal request created with 'pending' status	PASS
TC-014	Admin approve withdrawal	Status updated to 'approved', balance deducted	PASS
TC-015	JWT access token expiry	Refresh token cookie used to obtain new access token	PASS
TC-016	Logout clears all tokens	Cookies cleared, refresh token deleted from MongoDB	PASS
TC-017	Admin post announcement	Announcement saved, visible to writers on dashboard	PASS
TC-018	Change password with wrong current	400 error: current password verification failed	PASS

Table 6-2 Function Testing Results — All 18 Test Cases PASS

Form Route	Controller Method	Fields
POST /auth/login	authController.login	email, password
POST /auth/register	authController.register	username, email, password, confirmPassword
POST /auth/change-password	authController.changePasswor d	currentPassword, newPassword, confirmNew
POST /articles	articleController.create	title, body, categoryId, topicId, featuredImage, tags, shortDescription
POST /admin/categories	adminController.createCategor y	name, topics[], rpmGroup
POST /admin/topics	adminController.createTopic	name, categoryId, slug, rpmGroup
POST /admin/rpm	adminController.setRPM	rpmLevel, rate_per_1000
POST /profile/withdraw	profileController.withdraw	amount, method, accountDetails
POST /admin/announcements	adminController.postAnnounce ment	title, description, category

Table 6-1 Form Handling Reference

Security Testing Checklist

Security Check	Status	Implementation Detail
Passwords hashed with bcrypt	DONE	bcrypt.hash(password, 10) — 10 salt rounds on every save
Plaintext passwords never stored	DONE	Pre-save Mongoose hook hashes before any DB write
JWT secret is 256-bit random string	DONE	process.env.JWT_SECRET — never hardcoded in source
Access tokens short-lived (15 min)	DONE	expiresIn: '15m' in jwt.sign() options
Refresh tokens stored in MongoDB	DONE	User.refreshToken field — instantly deleteable on logout
HTTP-only cookies for JWT tokens	DONE	httpOnly: true prevents JavaScript access to tokens
Content Security Policy set	DONE	helmet.contentSecurityPolicy() with strict directives
X-Frame-Options: DENY set	DONE	Prevents all clickjacking attacks via Helmet
Environment variables for all secrets	DONE	.env file excluded by .gitignore — never committed
Error details hidden in production	DONE	NODE_ENV=production hides stack traces from users
Server-side validation on all inputs	DONE	Validation middleware runs before every POST handler
XSS prevention in EJS templates	DONE	<%= %> auto-escapes all HTML entities by default
NoSQL injection prevented	DONE	Mongoose parameterized queries prevent operator injection
Admin routes protected by RBAC	DONE	adminGuard returns 403 for any non-admin user

Table 6-3 Security Testing Checklist — All Critical Checks: DONE

Chapter 7

7 Conclusions & Future Work

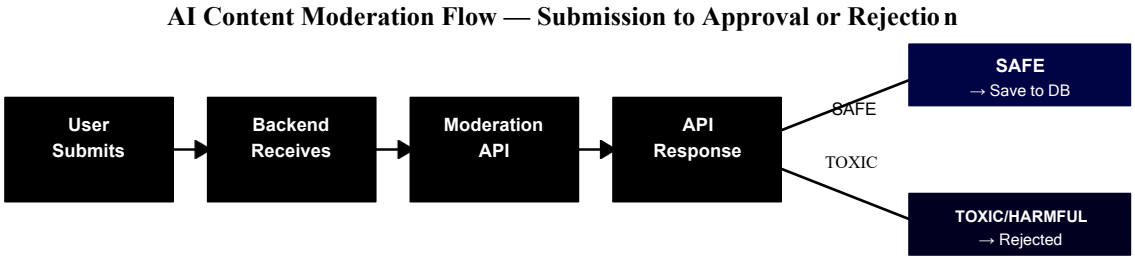


Figure 7-1 AI Content Moderation Flow — Submission to Approval or Rejection

In this project, we have investigated and developed a full-featured multi-vendor earning blog platform integrated with AI-powered content moderation. The system successfully demonstrates enterprise-grade authentication using JWT with refresh tokens, role-based access control, server-side rendering with EJS, and automated content safety using an AI moderation API. The platform is built on Node.js + Express.js with MongoDB Atlas as the cloud database, following the MVC architectural pattern throughout.

The project met all Priority 1 functional requirements: user registration and authentication, article creation with AI moderation, admin panel with category/topic/RPM management, writer earnings dashboard with blog analytics, withdrawal request management, contact messaging, and platform announcements. The architecture was designed with production scalability in mind using stateless JWT authentication and modular route organization.

There could be several improvements possible. Some of the ideas for future development:

Idea 1 — Advanced Earning Analytics: The Blog Analytics Dashboard is already implemented. Future: extend with custom date range filtering, per-category earning breakdown, and reader geographic demographics using MongoDB Atlas Charts.

Idea 2 — Automated Payout System: Withdrawal requests and admin approval are already implemented. Future: integrate Pakistani payment gateways (JazzCash, Easypaisa) for automated payment processing without manual admin intervention.

Idea 3 — Image Cloud Storage: Create Article form supports featured image upload. Future: migrate from local storage to Cloudinary or AWS S3 for cloud-based image hosting with automatic optimization, CDN delivery, and thumbnail generation.

Idea 4 — Email Notifications: Transactional emails for withdrawal approved/declined, new comments, and weekly earnings summaries using Nodemailer with SendGrid SMTP integration.

Idea 5 — Full-Text Search: Full-text search across article title, body, and tags using MongoDB Atlas Search or Elasticsearch for improved content discovery.

Idea 6 — Writer Public Profiles: Public writer profile pages with article portfolio, total view count, earnings rank, bio, and follower/following system.

Idea 7 — Mobile Application: Native iOS/Android app using React Native, consuming the existing REST API endpoints, providing writers with mobile notifications and on-the-go article management.

Idea 8 — Cloud Deployment & CI/CD: Production deployment to Railway or Render with automated CI/CD pipeline from GitHub — testing, linting, and deployment on every push to main branch.

Data Models

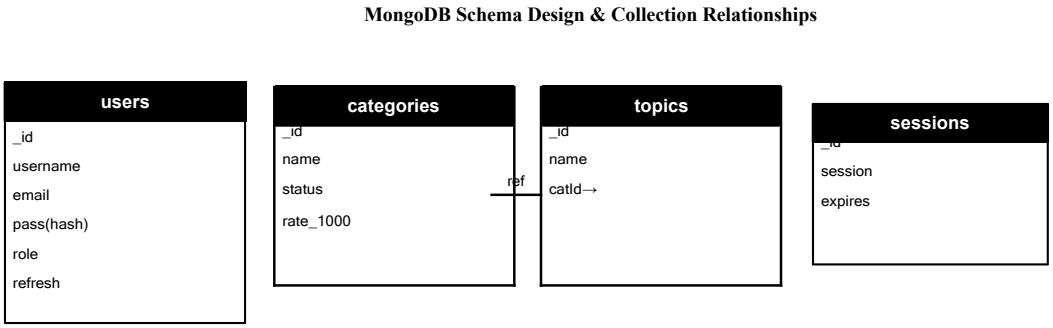


Figure 8-1 MongoDB Schema Design & Collection Relationships

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique document identifier
username	String	required, unique, trim, min:3, max:30	Public display name for the writer
email	String	required, unique, lowercase, trim	Primary login identifier
password	String	required, minlength:8	bcrypt hash — never stored as plaintext
role	String	enum:['admin','user'], default:'user'	Controls which routes and features are accessible
refreshToken	String	optional, nullable	Current JWT refresh token — deleted on logout
bio	String	maxlength:500, optional	Writer profile description shown on public profile
totalEarnings	Number	default:0, min:0	Cumulative earnings balance in USD
createdAt	Date	auto (timestamps:true)	Account creation timestamp
updatedAt	Date	auto (timestamps:true)	Last modification timestamp

Table 7-1 User Model Schema — Fields, Types and Constraints

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique identifier
name	String	required, unique, trim	Category display name (e.g. Finance & Business)
status	String	enum:['active','inactive'], default:'active'	Visibility control — inactive hidden from writers
rate_per_1000	Number	required, min:0	USD earnings per 1000 article views
description	String	maxlength:200, optional	Category description shown to writers

Table 7-2 Category Model Schema

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique identifier
name	String	required, trim	Topic display name (e.g. Personal Finance)
slug	String	required, unique	URL-friendly identifier (e.g. personal-finance)
categoryId	ObjectId	ref:'Category', required	Parent category ObjectId reference (Foreign Key)
rpmGroup	String	enum:['low','normal','medium','high']	RPM level that determines earning rate
articleCount	Number	default:0	Number of articles published under this topic

Table 7-3 Topic Model Schema

Variable	Example Value	Security Level
MONGO_URI	mongodb+srv://user:pass@cluster.mongodb.net/blogdb	Critical — full database access if leaked
JWT_SECRET	random 256-bit hex string	Critical — allows token forgery if exposed
JWT_REFRESH_SECRET	different 256-bit hex string	Critical — allows session bypass if exposed
SESSION_SECRET	random string for express-session	High — session hijacking risk if leaked
MODERATION_API_KEY	key_abc123xyz	Medium — API quota abuse if leaked
PORT	3000	Low — operational preference only
NODE_ENV	production / development	Medium — controls error verbosity and security flags

Table 7-4 Environment Configuration Variables and Security Levels

System Security Architecture

This section documents the complete security architecture implemented across the Blog Website platform. Security was treated as a first-class concern from the initial design phase, following the principle of Defense in Depth — multiple overlapping layers so that if one layer fails, the others still protect the system.

Layer 1 — Content Security Policy (CSP): Helmet middleware sets strict HTTP response headers. This prevents XSS attacks by blocking inline scripts and restricting script sources to the same origin plus explicitly whitelisted CDN domains. The frame-ancestors: none directive prevents clickjacking.

Layer 2 — JWT Authentication: Every protected route is guarded by authMiddleware which extracts the JWT from the HTTP-only cookie, calls jwt.verify() with the 256-bit secret key, and on failure redirects to the login page. Access tokens carry a 15-minute expiry. The refresh token is stored in MongoDB enabling instant revocation by deleting the DB record.

Layer 3 — Role-Based Access Control: The adminGuard middleware checks req.user.role === 'admin' on every admin route. Normal users receive 403 Forbidden regardless of how they discovered the URL. The role is stored both in MongoDB and embedded in the JWT payload.

Layer 4 — Input Validation: Server-side validation middleware runs before every POST route handler, regardless of any client-side validation. Fields checked for presence, format, type, and length. EJS's <%= %> syntax auto-escapes all HTML entities preventing stored XSS.

Layer 5 — AI Content Moderation: All user-submitted article content is screened through a third-party moderation API before any database write. The gate is absolute — content can only enter the database if it receives a SAFE classification. TOXIC content is blocked immediately.

References

[1] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

[2] OWASP Foundation (2021). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

[3] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force (IETF).

[4] Provos, N., & Mazieres, D. (1999). A future-adaptable password scheme. USENIX Annual Technical Conference.

[5] MongoDB Inc. (2023). MongoDB Documentation: Schema Design Patterns. https://www.mongodb.com/docs/

[6] Node.js Foundation (2023). Node.js Documentation v18 LTS. https://nodejs.org/en/docs/

[7] Express.js (2023). Express 4.x API Reference. https://expressjs.com/en/4x/api.html

[8] Mongoose (2023). Mongoose v7 Documentation. https://mongoosejs.com/docs/

[9] Cloudflare (2022). Content Security Policy: An Introduction to CSP Headers. Cloudflare Blog.

[10] MDN Web Docs (2023). HTTP cookies — Using HTTP cookies. https://developer.mozilla.org/

[11] GitHub (2023). Node.js Best Practices — goldbergyoni. https://github.com/goldbergyoni/nodebestpractices

[12] Reenskaug, T. (1979). THING-MODEL-VIEW-EDITOR: an Example from a planningsystem. Xerox PARC.

[13] Tailwind CSS (2023). Tailwind CSS Documentation. https://tailwindcss.com/docs

[14] EJS (2023). Embedded JavaScript Templating Documentation. https://ejs.co/

[15] W3C (2021). Content Security Policy Level 3. W3C Working Draft. https://www.w3.org/TR/CSP3/

[16] Bhargavan, K., et al. (2016). Transcript Collision Attacks: Breaking Auth in TLS, IKE, SSH. NDSS Symposium.

[17] Hunt, T. (2017). HTTPS is not enough — password security and bcrypt. Troy Hunt Blog.

[18] Hardt, D. (2012). The OAuth 2.0 Authorization Framework. RFC 6749. IETF.

Appendix A: List of Components

All major software components and npm packages used in the development of the Multi Vendor Earning Blog Site with AI platform.

Package	Version	Purpose
express	^4.18.2	Web framework — routing, middleware, HTTP handling
mongoose	^7.4.0	MongoDB ODM — schema definition, validation, query helpers
jsonwebtoken	^9.0.0	JWT creation and verification for authentication tokens
bcrypt	^5.1.0	Adaptive password hashing with configurable salt rounds
express-session	^1.17.3	Server-side session management for flash messages
connect-mongo	^5.0.0	MongoDB-backed session store for express-session
helmet	^7.0.0	HTTP security headers including Content Security Policy (CSP)
dotenv	^16.3.1	Load .env environment variables into process.env
ejs	^3.1.9	Embedded JavaScript template engine for server-side rendering
cors	^2.8.5	Cross-Origin Resource Sharing middleware configuration
morgan	^1.10.0	HTTP request logger middleware for development debugging
nodemon	^3.0.1	Auto-restart development server on source file changes
multer	^1.4.5	Multipart form data handling for featured image uploads
slugify	^1.6.6	URL-friendly slug generation from article/topic titles
express-validator	^7.0.1	Input validation and sanitization middleware library

Table A-1 npm Package List

Appendix B: Project Timeline

15-week project milestone plan distributed among three team members. Member 1: authentication & system architecture. Member 2: database, admin module, API integration. Member 3: frontend EJS templates, CSS styling, client-side JavaScript.

No.	Starting Week	Description of Milestone	Duration
1	Week 1	Project Setup —– Environment configuration, Git repository initialization, Node.js project setup, MVC folder structure definition,initial package installation and .env configuration.	1 Week
2	Week 2	Database Design — Mongoose schema design for User, Category, Topic, Article, and Withdrawal models. ER diagram creation, MongoDB Atlas cloud setup and connection testing.	1 Week
3	Week 3	Auth — Registration — User registration controller, bcrypt password hashing (10 rounds), server-side input validation middleware, email and username uniqueness checks.	1 Week
4	Week 4	Auth —– Login & JWT —– JWT access token generation (15 min expiry), refresh token system (7 days), HTTP-only cookie setup, authController.login implementation and testing.	1 Week
5	Week 5	Auth — Middleware & RBAC — authMiddleware (JWT verification), adminGuard (role checking returns 403), smart redirect logic, complete route protection for all modules.	1 Week
6	Week 6	Admin Module — Admin dashboard with stats aggregation, category CRUD with topic sub-forms, RPM rate configuration system, category status toggle (active/inactive).	1 Week
7	Week 7	Frontend —– EJS Templates —– Navbar and footer partials, home page, login/register forms, Author Panel dashboard layout, flash messaging integration, Tailwind CSS responsive design.	1 Week
8	Week 8	Article Creation & AI Moderation — Article submission form with image upload (Multer), third-party AI moderation API integration, SAFE/TOXIC/HARMFUL classification handling.	1 Week
9	Week 9	Frontend —– Dynamic Rendering —– Dynamic topic dropdown (data-attribute JSON pattern), Blog Analytics dashboard charts (Chart.js), performance trend visualization, mobile menu.	1 Week
10	Week 10	Earning System —– RPM Management page, view counter implementation, earnings calculation formula (views/1000 × RPMrate), Earnings Dashboard with metric cards.	1 Week
11	Week 11	Withdrawal & Admin Features — Withdraw Funds form, admin Withdrawal Requests management (approve/decline), Contact Messages module, Announcements system.	1 Week
12	Week 12	Security Hardening — Helmet CSP headers configuration, SRI for CDN scripts, full input sanitization audit, CORS setup, environmentvariable security review.	1 Week

No.	Starting Week	Description of Milestone	Duration
13	Week 13	Testing —- Unit & Integration —- All 18 test cases (TC-001 to TC-018) executed. Security checklist completed. Bug fixes from regression testing. Performance optimization.	1 Week
14	Week 14	Documentation & Report Writing —- FYP report writing, all UML/class/ER/state/use-case diagrams, figures, tables, appendices,table of contents, list of figures.	1 Week
15	Week 15	Final Review & Submission — Complete code review, security audit, report proofreading, PDF generation, CD/DVD preparation, supervisor sign-off, final submission.	1 Week

Table B-1 15-Week Project Timeline

** A Gantt chart can be substituted for this table if preferred by the department.*

Appendix C: User Manual

How a person will use the Multi Vendor Earning Blog Site with AI. The platform has two distinct user types: Writers (registered users who publish content and earn revenue) and Administrators (who manage the entire platform configuration).

C.1 Getting Started — Writer Registration

1. Open your browser and navigate to the platform URL.

2. Click the Register button in the top navigation bar.

3. Fill in the registration form: username (3–30 alphanumeric characters and underscores only), email address, and password (minimum 8 characters with at least one number). Confirm your password.

4. Click Submit. If all inputs are valid and your email is not already registered, you will be redirected to the login page with a green success message.

5. Login with your email and password. Upon successful login, you will be redirected to your personal Author Panel.

C.2 Writing and Publishing an Article

1. From your Author Panel, click Create Post in the left sidebar.

2. Enter your Blog Title (10–200 characters). Upload a Featured Image (optional). Add Tags separated by commas (e.g. security, nodejs, tips).

3. Write a Short Description (article summary for search results).

4. Scroll down to write your Full Content in the rich text area (minimum 100 characters).

5. Click Publish. The AI Moderation API screens your content automatically. If SAFE: article is published and you see a green success message. If TOXIC or HARMFUL: article is blocked and you receive an error message asking you to revise the content.

6. View your published articles by clicking Your Posts in the sidebar. The table shows each article's title, date, views, and likes count.

C.3 Viewing Your Analytics and Earnings

1. Click Dashboard in your Author Panel sidebar to access the Blog Analytics Dashboard.

2. The dashboard shows five metric cards: Blogs, Views, Likes, Shares, and Comments — each showing total, this month, and today counts.

3. Click Earnings in the sidebar to access the Earnings Dashboard. This shows: Total Earnings ($), Monthly Earnings, Today's Earnings, and Available Balance.

4. Earnings formula: Earnings = (Article Views ÷ 1000) × RPM Rate. For Medium RPM ($10): 5,000 views earns $50.00.

C.4 Withdrawing Your Earnings

1. Click Withdraw in your Author Panel sidebar.

2. Enter the Amount ($) to withdraw. The minimum withdrawal is $10.

3. Select your Withdrawal Method from the dropdown (bank transfer, JazzCash, Easypaisa, etc.).

4. Enter your Account Details (account number or payment ID) and click Submit Withdrawal Request.

5. Requests are processed by the admin within 3–5 business days. Ensure your account information is accurate.

C.5 Admin Panel — Managing the Platform

1. Admin Panel sidebar: Dashboard, Users, Categories, Messages, Earnings, Withdrawal Requests, Settings, RPM Price, categoryadd, Announcements.

2. Navigate to RPM Price to set earning rates. Configure RPM Level (Low/Normal/Medium/High) and Rate per 1000 Views ($).

3. Navigate to categoryadd to create categories. Enter Category Name, add Topics (each with Title, Slug, and RPM Group), click Save Category.

4. Navigate to Withdrawal Requests to process writer payouts. Click Approve (green) or Decline (red) for each pending request.

5. Navigate to Messages to view and manage contact messages from writers (shows Unread/Read/Total counts).

6. Navigate to Announcements to post platform updates: enter Update Title, Description, select Category, click Post Update.

7. Navigate to Settings to change the admin password. Enter current password (for verification), new password, and confirm new password, then click Update.